

Fugacity from Cubic Equations of State

Author: Edward Maginn, CBE 20260

This notebook computes the **fugacity coefficient** ϕ and **fugacity** $f = \phi P$ for a real fluid using the van der Waals (vdW), Soave-Redlich-Kwong (SRK), and Peng-Robinson (PR) equations of state. When the cubic EOS yields two physical roots (liquid and vapor), the notebook computes both fugacities and identifies the **thermodynamically stable phase** as the one with the lower fugacity.

Background: $\ln \phi$ Formulas

Starting from the general formula

$$\ln \phi = \int_0^P \frac{Z - 1}{P} dP$$

and evaluating analytically for each EOS, we obtain closed-form expressions. Define the dimensionless parameters

$$A \equiv \frac{aP}{(RT)^2}, \quad B \equiv \frac{bP}{RT}$$

where a includes the $\alpha(T)$ correction for SRK and PR.

van der Waals:

$$\ln \phi = (Z - 1) - \ln(Z - B) - \frac{A}{Z}$$

Soave-Redlich-Kwong (SRK):

$$\ln \phi = (Z - 1) - \ln(Z - B) - \frac{A}{B} \ln \left(1 + \frac{B}{Z} \right)$$

Peng-Robinson (PR):

$$\ln \phi = (Z - 1) - \ln(Z - B) - \frac{A}{2\sqrt{2}B} \ln \left(\frac{Z + (1 + \sqrt{2})B}{Z + (1 - \sqrt{2})B} \right)$$

The **phase stability rule**: when both vapor (Z^V) and liquid (Z^L) roots exist, nature selects the phase with the **lower fugacity** $f = \phi P$.

Python Solver

Select the EOS, fluid, temperature, and pressure using the widgets below, then read off the fugacity of each root and the predicted stable phase.

```
# Install CoolProp if it is missing (e.g., on Google Colab)
try:
    import CoolProp
except ImportError:
    %pip install -q CoolProp

import numpy as np
import matplotlib.pyplot as plt
import ipywidgets as widgets
from IPython.display import display, clear_output
import CoolProp.CoolProp as CP

fluid_dict = {
    'Methane':      'Methane',
    'Ethane':       'Ethane',
    'Propane':      'Propane',
    'n-Hexane':     'n-Hexane',
    'Nitrogen':     'Nitrogen',
    'Oxygen':       'Oxygen',
    'Ammonia':      'Ammonia',
    'Carbon Dioxide': 'CarbonDioxide',
    'Water':        'Water',
    'Argon':        'Argon',
}

R_Lbar = 0.0831446 # L·bar/(mol·K)
R_J     = 8.31446  # J/(mol·K)

# — EOS parameter builders
↪ _____

def get_eos_params(eos, fluid, T, P, Tc, Pc, omega):
    """Return (a, b, da_dT, roots_V) for the chosen EOS."""
    Tr = T / Tc
    if eos == 'vdW':
        a = 27 * (R_Lbar * Tc)**2 / (64 * Pc)
        b = R_Lbar * Tc / (8 * Pc)
        da_dT = 0.0
        C2 = -(b + R_Lbar * T / P)
        C1 = a / P
        C0 = -a * b / P
    elif eos == 'SRK':
        ac = 0.42748 * R_Lbar**2 * Tc**2 / Pc
```

```

        b      = 0.08664 * R_Lbar * Tc / Pc
        m      = 0.480 + 1.574 * omega - 0.176 * omega**2
        alpha  = (1 + m * (1 - Tr**0.5))**2
        a      = ac * alpha
        da_dT  = -ac * m * (alpha / (T * Tc))**0.5
        C2     = -(R_Lbar * T / P)
        C1     = a / P - b**2 - R_Lbar * T * b / P
        C0     = -(a * b / P)
    else:      # PR
        ac     = 0.45724 * R_Lbar**2 * Tc**2 / Pc
        b      = 0.07780 * R_Lbar * Tc / Pc
        m      = 0.37464 + 1.54226 * omega - 0.26992 * omega**2
        alpha  = (1 + m * (1 - Tr**0.5))**2
        a      = ac * alpha
        da_dT  = -ac * m * (alpha / (T * Tc))**0.5
        C2     = b - R_Lbar * T / P
        C1     = a / P - 2 * b * R_Lbar * T / P - 3 * b**2
        C0     = b**3 + R_Lbar * T / P * b**2 - a * b / P

    r = np.roots([1.0, C2, C1, C0])
    roots_V = np.sort(r[np.abs(r.imag) < 1e-9].real)
    roots_V = roots_V[roots_V > b]
    return a, b, da_dT, roots_V

# — Fugacity coefficient formulas
↪ —————

def ln_phi(eos, Z, A, B):
    """Evaluate ln φ for the given EOS and compressibility
    ↪ factor Z."""
    if eos == 'vdW':
        return (Z - 1) - np.log(Z - B) - A / Z
    elif eos == 'SRK':
        return (Z - 1) - np.log(Z - B) - (A / B) * np.log(1 + B
    ↪ / Z)
    else:      # PR
        sq2 = np.sqrt(2)
        return (Z - 1) - np.log(Z - B) - A / (2 * sq2 * B) *
    ↪ np.log(
        (Z + (1 + sq2) * B) / (Z + (1 - sq2) * B)
    )

# — Main solver/plotter
↪ —————

def run(eos_choice, fluid_name, T_val, T_unit, P_val, P_unit):

```

```

# Unit conversions
T = T_val + 273.15 if T_unit == '°C' else (T_val - 32) * 5/9
↪ + 273.15 if T_unit == '°F' else T_val
conv = {'bar': 1, 'atm': 1.01325, 'Pa': 1e-5, 'MPa': 10,
↪ 'psi': 1/14.5038}
P = P_val * conv[P_unit]
if T <= 0 or P <= 0:
    print("Temperature and pressure must be positive.");
    ↪ return

fluid = fluid_dict[fluid_name]
Tc = CP.PropsSI('TCRIT', fluid)
Pc = CP.PropsSI('PCRIT', fluid) / 1e5 # bar
omega = CP.PropsSI('ACENTRIC', fluid)

a, b, da_dT, roots_V = get_eos_params(eos_choice, fluid, T,
↪ P, Tc, Pc, omega)

A = a * P / (R_Lbar * T)**2
B = b * P / (R_Lbar * T)
V_ig = R_Lbar * T / P

# — Print header
↪
print("=" * 72)
print(f" EOS: {eos_choice} | Fluid: {fluid_name}")
print(f" Tc = {Tc:.2f} K Pc = {Pc:.2f} bar ω =
↪ {omega:.4f}")
print(f" T = {T:.2f} K P = {P:.3f} bar")
print(f" A = {A:.5f} B = {B:.5f}")
print(f" V_ideal = {V_ig:.4f} L/mol")
print("=" * 72)

fugacities = [] # (label, V, Z, lnphi, f)

for i, V in enumerate(roots_V):
    Z = P * V / (R_Lbar * T)
    lnphi = ln_phi(eos_choice, Z, A, B)
    phi = np.exp(lnphi)
    f = phi * P

    n = len(roots_V)
    if n == 1:
        label = "Vapor" if T >= Tc or V > 5 * b else
↪ "Liquid"
    else:
        if i == 0: label = "Liquid root"
        elif i == 1: label = "Unstable root (discard)"

```

```

        else:
            label = "Vapor root"

    fugacities.append((label, V, Z, lnp, phi, f))

    print(f"\n  [{label}]\n")
    print(f"    V      = {V:.5f} L/mol")
    print(f"    Z      = {Z:.5f}")
    if "Unstable" in label:
        print("      (fugacity not meaningful for unstable
          ↪ root - skip)")
        continue
    print(f"    ln φ    = {lnp:.5f}")
    print(f"    φ      = {phi:.5f}")
    print(f"    f      = {f:.4f} bar")

# — Phase stability verdict
↪
physical = [(lbl, V, Z, lnp, phi, f)
            for (lbl, V, Z, lnp, phi, f) in fugacities
            if "Unstable" not in lbl]

print("\n" + "-" * 72)
if len(physical) == 2:
    lbl_L, _, _, _, f_L = physical[0]
    lbl_V, _, _, _, f_V = physical[1]
    print(f"  PHASE STABILITY ANALYSIS:")
    print(f"    f^L = {f_L:.4f} bar    (liquid root)")
    print(f"    f^V = {f_V:.4f} bar    (vapor root)")
    if abs(f_L - f_V) / max(f_L, f_V) < 1e-4:
        print("      ↪ f^L ≈ f^V : VLE condition satisfied -
          ↪ system is on the saturation curve.")
    elif f_L < f_V:
        print("      ↪ f^L < f^V : LIQUID is the stable
          ↪ phase at this T and P.")
    else:
        print("      ↪ f^V < f^L : VAPOR is the stable phase
          ↪ at this T and P.")
elif len(physical) == 1:
    print(f"  Single physical root - {physical[0][0]}
      ↪ phase.")
    print(f"  f = {physical[0][5]:.4f} bar,  φ =
      ↪ {physical[0][4]:.5f}")
print("=" * 72)

# — Plot
↪
V_max = max(V_ig * 1.5, 10 * b)
if len(roots_V) == 3:

```

```

    V_max = max(V_max, roots_V[2] * 1.3)
    V_arr = np.linspace(b * 1.005, V_max, 3000)

    if eos_choice == 'vdW':
        P_iso = R_Lbar * T / (V_arr - b) - a / V_arr**2
    elif eos_choice == 'SRK':
        P_iso = R_Lbar * T / (V_arr - b) - a / (V_arr * (V_arr +
↪ b))
    else:
        P_iso = R_Lbar * T / (V_arr - b) - a / (V_arr**2 +
↪ 2*b*V_arr - b**2)

    fig, axes = plt.subplots(1, 2, figsize=(13, 5.5))

    # Left panel: P-V diagram
    ax = axes[0]
    ax.plot(V_arr, P_iso, 'b-', lw=2, label=f'{eos_choice}
↪ isotherm (T = {T:.1f} K)')
    ax.axhline(P, color='k', ls=':', lw=1.5, label=f'P = {P:.2f}
↪ bar')

    colors_root = {'Liquid root': '#d62728', 'Vapor root':
↪ '#1f77b4',
                   'Unstable root (discard)': 'lightgray'}
    for (lbl, V, Z, lnp, phi, f) in fugacities:
        color = colors_root.get(lbl, 'green')
        marker = 'x' if 'Unstable' in lbl else 'o'
        tag = '' if 'Unstable' in lbl else f' f={f:.3f} bar'
        ax.plot(V, P, marker=marker, ms=9, color=color,
↪ zorder=5, label=lbl + tag)

    ax.set_xlim(0, V_max)
    ax.set_ylim(bottom=0, top=max(P * 2.2, Pc * 1.6))
    ax.set_xlabel('Molar volume, $V$ (L/mol)', fontsize=12)
    ax.set_ylabel('Pressure, $P$ (bar)', fontsize=12)
    ax.set_title(f'{eos_choice} isotherm - {fluid_name}',
↪ fontsize=12, fontweight='bold')
    ax.legend(fontsize=8.5, loc='upper right')
    ax.grid(True, ls='--', alpha=0.5)

    # Right panel: fugacity bar chart
    ax2 = axes[1]
    bar_labels = []
    bar_vals = []
    bar_colors = []
    for (lbl, V, Z, lnp, phi, f) in fugacities:
        if 'Unstable' in lbl:
            continue

```

```

        bar_labels.append(lbl.replace(' root', ''))
        bar_vals.append(f)
        bar_colors.append('#d62728' if 'Liquid' in lbl else
↪ '#1f77b4' if 'Vapor' in lbl else '#2ca02c')

    # Also add f = P reference
    bar_labels.append('Ideal (f = P)')
    bar_vals.append(P)
    bar_colors.append('lightgray')

    bars = ax2.bar(bar_labels, bar_vals, color=bar_colors,
↪ edgecolor='k', width=0.4)
    for bar, val in zip(bars, bar_vals):
        ax2.text(bar.get_x() + bar.get_width()/2,
↪ bar.get_height() + 0.01 * max(bar_vals),
                f'{val:.3f}', ha='center', va='bottom',
                ↪ fontsize=10, fontweight='bold')

    if len(physical) == 2:
        stable_f = min(physical[0][5], physical[1][5])
        ax2.axhline(stable_f, color='green', ls='--', lw=1.5,
↪ label='Stable phase (lower f)')
        ax2.legend(fontsize=9)

    ax2.set_ylabel('Fugacity, $f$ (bar)', fontsize=12)
    ax2.set_title('Fugacity comparison', fontsize=12,
↪ fontweight='bold')
    ax2.grid(True, axis='y', ls='--', alpha=0.5)
    ax2.set_ylim(bottom=0)

    plt.suptitle(f'{fluid_name} at T = {T:.1f} K, P = {P:.2f}
↪ bar    [{eos_choice}]',
                fontsize=13, y=1.01)
    plt.tight_layout()
    plt.show()

# — Widgets
↪
style = {'description_width': 'initial'}
lay    = widgets.Layout

eos_dd  = widgets.Dropdown(options=['Peng-Robinson', 'SRK',
↪ 'vdW'], value='Peng-Robinson',
                            description='EOS:', style=style)
fluid_dd = widgets.Dropdown(options=list(fluid_dict.keys()),
↪ value='Propane',

```

```

description='Fluid:', style=style)

T_in = widgets.FloatText(value=300,
    ↪ description='Temperature:', style=style,
    ↪ layout=lay(width='200px'))
T_unit = widgets.Dropdown(options=['K', '°C', '°F'], value='K',
    ↪ layout=lay(width='80px'))
P_in = widgets.FloatText(value=10, description='Pressure:',
    ↪ style=style, layout=lay(width='200px'))
P_unit = widgets.Dropdown(options=['bar', 'atm', 'Pa', 'MPa',
    ↪ 'psi'], value='bar', layout=lay(width='80px'))

ui = widgets.VBox([eos_dd, fluid_dd,
    widgets.HBox([T_in, T_unit]),
    widgets.HBox([P_in, P_unit])])
out = widgets.interactive_output(run, {
    'eos_choice': eos_dd, 'fluid_name': fluid_dd,
    'T_val': T_in, 'T_unit': T_unit,
    'P_val': P_in, 'P_unit': P_unit,
})

display(ui, out)

```

```

# Static preview: results for the default inputs above.
# The output is saved so the course website can show it.
# To try other inputs, use the controls above, or open
# this notebook in Colab (badge on the course website).
run(eos_choice=eos_dd.value, fluid_name=fluid_dd.value,
    ↪ T_val=T_in.value, T_unit=T_unit.value,
    P_val=P_in.value, P_unit=P_unit.value)

```

```

=====
EOS: Peng-Robinson | Fluid: Propane
Tc = 369.89 K   Pc = 42.51 bar   ω = 0.1521
T = 300.00 K   P = 10.000 bar
A = 0.18370    B = 0.02256
V_ideal = 2.4943 L/mol
=====

```

```

[Liquid root]
V      = 0.08670 L/mol
Z      = 0.03476
ln φ   = -0.17370
φ      = 0.84055
f      = 8.4055 bar

```

```
[Unstable root (discard)]
V      = 0.31927 L/mol
Z      = 0.12800
(fugacity not meaningful for unstable root – skip)
```

```
[Vapor root]
V      = 2.03209 L/mol
Z      = 0.81468
ln  $\phi$  = -0.17179
 $\phi$     = 0.84216
f      = 8.4216 bar
```

PHASE STABILITY ANALYSIS:

$f^L = 8.4055$ bar (liquid root)

$f^V = 8.4216$ bar (vapor root)

→ $f^L < f^V$: LIQUID is the stable phase at this T and P.

Propane at T = 300.0 K, P = 10.00 bar [Peng-Robinson]

